

Procedure và Function



Giới thiệu

Trong PostgreSQL, **Procedure (Thủ tục)** và **Function (Hàm)** là hai khái niệm cốt lõi của lập trình phía máy chủ. Chúng giúp bạn đóng gói các câu lệnh SQL và logic nghiệp vụ (business logic) lại với nhau để tái sử dụng, tăng hiệu năng và bảo mật dữ liệu.

Mặc dù trông khá giống nhau, chúng có những mục đích sử dụng và đặc điểm rất khác biệt.

Function

Function

Mục đích chính của Function là tính toán và trả về một giá trị, hoặc một bảng.

Có hai dạng function:

- Built-in Functions (Hàm tích hợp sẵn).
- User-Defined Function (UDF - Hàm do người dùng định nghĩa).

Built-in Function

Built-in Function

PostgreSQL cung cấp một thư viện khổng lồ các Built-in Functions (Hàm tích hợp sẵn) để bạn sử dụng ngay lập tức mà không cần tự định nghĩa. Các hàm này được tối ưu hóa tối đa về hiệu năng và chia thành nhiều nhóm khác nhau.

Nhóm hàm xử lý Chuỗi

Dùng để cắt, nối, tìm kiếm hoặc biến đổi văn bản.

- `CONCAT(str1, str2, ...)` hoặc **toán tử ||**: Nối các chuỗi lại với nhau.
- `UPPER(str)` / `LOWER(str)`: Chuyển chữ hoa / chữ thường.
- `LENGTH(str)`: Đếm số ký tự của chuỗi.
- `SUBSTRING(str FROM start FOR length)`: Cắt chuỗi con.
- `REPLACE(str, from_str, to_str)`: Thay thế chuỗi.

Nhóm hàm Toán học

Dùng cho các phép tính số học từ cơ bản đến nâng cao.

- **ROUND(v NUMERIC, s INT)**: Làm tròn số v đến s chữ số thập phân.
- **CEIL(num) / FLOOR(num)**: Làm tròn lên / Làm tròn xuống số nguyên gần nhất.
- **ABS(num)**: Lấy giá trị tuyệt đối.
- **POWER(a, b)**: Tính a lũy thừa b.
- **RANDOM()**: Tạo một số ngẫu nhiên từ 0.0 đến 1.0.

Nhóm hàm Ngày tháng & Thời gian

Nhóm hàm cực kỳ quan trọng trong quản lý dữ liệu log, báo cáo, doanh thu.

- **NOW()** hoặc **CURRENT_TIMESTAMP**: Lấy ngày giờ hiện tại của hệ thống.
- **CURRENT_DATE**: Lấy ngày hiện tại (không có giờ).
- **EXTRACT(field FROM timestamp)**: Tách một thành phần (như YEAR, MONTH, DAY, HOUR) ra khỏi thời gian.
- **AGE(timestamp1, timestamp2)**: Tính khoảng cách thời gian giữa 2 mốc (thường dùng tính tuổi).

Nhóm hàm Điều kiện

Giúp bạn thực hiện logic "Nếu... Thì..." ngay trong câu lệnh SQL.

- **COALESCE(val1, val2, ..., valN)**: Trả về giá trị đầu tiên khác NULL trong danh sách. Rất hữu ích để thiết lập giá trị mặc định.
- **NULLIF(val1, val2)**: Trả về NULL nếu val1 = val2, ngược lại trả về val1 (thường dùng để tránh lỗi chia cho 0).

User-Defined Function

User-Defined Function

Trái ngược với Built-in Function (hàm có sẵn), User-Defined Function (UDF - Hàm do người dùng định nghĩa) là những hàm do chính bạn tự viết ra để giải quyết các bài toán nghiệp vụ đặc thù của dự án.

Trong PostgreSQL, UDF cực kỳ mạnh mẽ vì nó không chỉ hỗ trợ ngôn ngữ SQL thuần túy mà còn hỗ trợ PL/pgSQL (ngôn ngữ lập trình thủ tục của Postgres), thậm chí cả Python (plpython), Perl (plperl)...

Cú pháp

```
CREATE [OR REPLACE] FUNCTION tên_hàm (  
    tham_số_1 kiểu_dữ_liệu,  
    tham_số_2 kiểu_dữ_liệu  
)  
RETURNS kiểu_dữ_liệu_trả_về AS $$  
DECLARE  
    -- Nơi khai báo các biến (nếu dùng PL/pgSQL)  
    biến_tạm kiểu_dữ_liệu;  
BEGIN  
    -- Nơi viết logic xử lý code  
    RETURN giá_trị;  
END;  
$$ LANGUAGE ngôn_ngữ; -- ngôn ngữ thường dùng như sql, plpgsql
```

Hàm SQL thuần túy

Thích hợp cho các hàm ngắn, tính toán đơn giản, không có logic rẽ nhánh (if-else / loop). Loại hàm này có hiệu năng rất cao vì Postgres có thể tối ưu hóa nó trực tiếp vào câu lệnh gọi.

Ví dụ

```
-- Hàm tính giá sau khi giảm giá
CREATE OR REPLACE FUNCTION tinh_giam_gia(gia_goc NUMERIC, phan_tram_giam INT)
RETURNS NUMERIC AS $$
    SELECT gia_goc * (1 - phan_tram_giam / 100.0);
$$ LANGUAGE sql;

-- Cách dùng:
SELECT tinh_giam_gia(100000, 15); -- Kết quả: 85000
```

Hàm PL/pgSQL

Sử dụng khi bạn cần khai báo biến, dùng câu lệnh điều kiện IF-THEN-ELSE, vòng lặp FOR/WHILE, hoặc bắt lỗi (EXCEPTION).

Ví dụ

```
CREATE FUNCTION get_film_count(len_from int, len_to int)
RETURNS INT AS $$
    DECLARE
        film_count INT;
    BEGIN
        SELECT COUNT(*) INTO film_count
        FROM film WHERE length BETWEEN len_from AND len_to;
        -- trả kết quả
        RETURN film_count;
    END;
$$ LANGUAGE plpgsql;
```

Hàm trả về một Bảng dữ liệu

Thay vì trả về 1 giá trị đơn lẻ, hàm này trả về cả một tập dữ liệu (nhiều hàng, nhiều cột), hoạt động giống như một View động có tham số.

Ví dụ

```
-- Hàm lấy danh sách sản phẩm theo danh mục và có lượt đánh giá cao
CREATE OR REPLACE FUNCTION lay_san_pham_hot(danh_muc_id INT, diem_toi_thieu INT)
RETURNS TABLE(id INT, ten_sp TEXT, gia NUMERIC) AS $$
BEGIN
    RETURN QUERY
    SELECT sp.id, sp.ten, sp.gia
    FROM san_pham sp
    WHERE sp.danh_muc = danh_muc_id AND sp.danh_gia >= diem_toi_thieu;
END;
$$ LANGUAGE plpgsql;

-- Cách dùng (Xem nó như một bảng dữ liệu):
SELECT * FROM lay_san_pham_hot(3, 4);
```

Quản lý UDF

Xóa hàm: Để xóa một hàm, bạn phải chỉ rõ tên hàm và kiểu dữ liệu của các tham số đầu vào (vì Postgres hỗ trợ nạp chồng hàm - Function Overloading: các hàm trùng tên nhưng khác tham số).

```
DROP FUNCTION tinh_giam_gia(NUMERIC, INT);
```

Xem lại định nghĩa hàm: Nếu muốn xem lại nội dung code của một hàm đã tạo bằng SQL

```
SELECT prosrc FROM pg_proc WHERE proname = 'tên_hàm_viết_thường';
```

Procedure

Procedure

Procedure được giới thiệu từ phiên bản PostgreSQL 11. Mục đích chính của Procedure là thực hiện một chuỗi các hành động/nghiệp vụ phức tạp.

Đặc điểm

- **Không trả về giá trị theo cách thông thường:** Procedure không có mệnh đề RETURNS. Nếu muốn trả ra dữ liệu, bạn phải sử dụng các tham số đầu ra (INOUT hoặc OUT).
- **Quản lý được Transaction:** Đây là lý do Procedure ra đời. Bên trong Procedure, có thể chủ động gọi COMMIT hoặc ROLLBACK. Điều này cực kỳ hữu ích khi cần xử lý hàng triệu bản ghi theo từng đợt (batch) để tránh làm nghẽn bộ nhớ.
- **Cách gọi:** Sử dụng câu lệnh CALL.
- **Không thể nhúng vào SQL:** Bạn không thể viết `SELECT * FROM table WHERE id = CALL my_procedure()`

Cú pháp

```
CREATE [OR REPLACE] PROCEDURE tên_procedure(  
    tham_so_1 kiểu_dữ_liệu,  
    tham_so_2 kiểu_dữ_liệu  
)  
LANGUAGE plpgsql AS $$  
DECLARE  
    -- Khai báo biến ở đây (nếu có)  
    bien_tam INT;  
BEGIN  
    -- Logic xử lý SQL  
    INSERT INTO bảng ... ;  
    -- Có thể commit giữa chừng nếu cần  
    COMMIT;  
    UPDATE bảng ... ;  
END;  
$$;
```


Ví dụ

```
CREATE OR REPLACE PROCEDURE chuyen_tien(id_nguoi_gui INT, id_nguoi_nhan INT, so_tien NUMERIC)
LANGUAGE plpgsql AS $$
BEGIN
    -- 1. Trừ tiền người gửi
    UPDATE tai_khoan SET so_du = so_du - so_tien
    WHERE id = id_nguoi_gui;
    -- 2. Cộng tiền người nhận
    UPDATE tai_khoan SET so_du = so_du + so_tien
    WHERE id = id_nguoi_nhan;
    -- 3. Lưu lại lịch sử giao dịch thành công vào DB
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        -- Nếu có lỗi xảy ra (ví dụ: mất kết nối, sai ID...), rollback lại toàn bộ
        RAISE NOTICE 'Đã xảy ra lỗi, hệ thống tự động hoàn tác giao dịch.';
        ROLLBACK;
END;
$$;
```

Phân biệt Function và Procedure

Đặc điểm	Function (Hàm)	Procedure (Thủ tục)
Từ khóa khởi tạo	CREATE FUNCTION	CREATE PROCEDURE
Giá trị trả về	Bắt buộc (RETURNS <kiểu_dữ_liệu>)	Không có RETURNS (Dùng OUT nếu cần)
Cách gọi	SELECT function_name();	CALL procedure_name();
Quản lý Transaction	Không (Không được dùng COMMIT/ROLLBACK)	Có (Được phép dùng COMMIT/ROLLBACK)
Vị trí sử dụng	Trong các câu lệnh SQL (SELECT, WHERE...)	Phải đứng độc lập bằng lệnh CALL
Ứng dụng chính	Tính toán, biến đổi dữ liệu, làm trigger.	Chạy ngầm (Cronjob), xử lý dữ liệu lớn, chuyển tiền...

Bài tập

Thông tin

Cho CSDL sau:

`product_hierarchy(id, parent_id, level_text, level_name)`

`product_details(product_id, price, product_name, category_id,
segment_id, style_id)`

`sales(product_id, qty, discount, member, txn_id, txd_time)`

Bài tập

Câu 1: Viết một hàm nhận vào `product_id` và trả về tổng doanh thu của sản phẩm đó. Doanh thu của mỗi lượt bán được tính bằng công thức:

$$\text{qty} * \text{price} * (1 - \text{discount} / 100)$$

Câu 2: Viết một hàm nhận vào `txn_id` (Mã giao dịch). Kiểm tra xem giao dịch đó có phải của thành viên không (`member = true`).

Câu 3: Viết một hàm nhận vào mã danh mục `p_category_id`. Hàm này sẽ trả về một danh sách gồm: `product_name`, `price` và tổng số lượng (`qty`) đã bán được của các sản phẩm thuộc danh mục đó.